NAME:RISHIT V N S **SRN:PES1UG24AM225** **OS ORANGE PROBLEM 2**

Screenshot 1A: Output of ./test_objects showing all tests passing.

```
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B: find .pes/objects -type f showing the sharded directory structure.

```
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$
```

Screenshot 2A: Output of ./test_tree showing all tests passing.

```
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$
```

Screenshot 2B: Pick a tree object from find .pes/objects -type f and run xxd .pes/objects/XX/YYY... | head -20 to show the raw binary format.

```
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ find .pes/objects -type f
.pes/objects/0f/61005ed64f7ff4f2db535b797e872a111b8bb9fa33040b7b32e1ba193df811
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ xxd .pes/objects/9f/61005ede6ff7ff4f2db535b797e872a111b8bb9fa33040b7b32e1ba193df811 | head -20
xxd: .pes/objects/9f/61005ede6ff7ff4f2db535b797e872a111b8bb9fa33040b7b32e1ba193df811: No such file or directory
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ xxd .pes/objects/0f/61005ed64f7ff4f2db535b797e872a111b8bb9fa33040b7b32e1ba193df811
00000000: 7472 6565 2039 0074 6573 7420 7472 6565  tree 9.test tree
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$
```

Screenshot 3A: Run ./pes init, ./pes add file1.txt file2.txt, ./pes status — show the output.

```
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ echo "hello" > file1.txt
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ echo "world" > file2.txt
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ ./pes add file1.txt file2.txt
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  pes.c
  untracked:  tree.c
  untracked:  commit.h
  untracked:  temp
  untracked:  index.h
  untracked:  commit.c
  untracked:  pes.h
  untracked:  test_tree.c
  untracked:  README.md
  untracked:  Makefile
  untracked:  test_objects.c
  untracked:  temp.c
  untracked:  .gitignore
  untracked:  index.c
  untracked:  tree.h
  untracked:  test_sequence.sh
  untracked:  object.c

rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$
```

Screenshot 3B: cat .pes/index showing the text-format index with your entries.

```
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776755467 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776755473 6 file2.txt
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$
```

Screenshot 4A: Output of ./pes log showing three commits with hashes, authors, timestamps, and messages.

```
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ echo "Hello" > hello.txt
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ ./pes add hello.txt
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ ./pes commit -m "Initial commit"
Committed: 12ffdff0d198... Initial commit
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ echo "World" >> hello.txt
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ ./pes add hello.txt
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ ./pes commit -m "Add world"
Committed: 2a2005348f9f... Add world
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ echo "Goodbye" > bye.txt
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ ./pes add bye.txt
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ ./pes commit -m "Add farewell"
Committed: 958591714ba4... Add farewell
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ ./pes log
commit 958591714ba40deea87807a2ff97615896fb83446b9ab6e78fb8e2dcfcfaf863
Author: PES User <pes@localhost>
Date:   1776756549

    Add farewell

commit 2a2005348f9fb90b32effbeb43827d58deba75694155961e94af5fdb229e5427
Author: PES User <pes@localhost>
Date:   1776756532

    Add world

commit 12ffdff0d1986b23cc6e62f1c9a069c334e3fea59d53a70b99d542e523b60b5f
Author: PES User <pes@localhost>
Date:   1776756473

    Initial commit

rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$
```

Screenshot 4B: find .pes -type f | sort showing object store growth after three commits.

```
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/12/ffdff0d1986b23cc6e62f1c9a069c334e3fea59d53a70b99d542e523b60b5f
.pes/objects/2a/2005348f9fb90b32effbeb43827d58deba75694155961e94af5fdb229e5427
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/95/8591714ba40deea87807a2ff97615896fb83446b9ab6e78fb8e2dcfcfaf863
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$
```

Screenshot 4C: cat .pes/refs/heads/main and cat .pes/HEAD showing the reference chain.

```
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ cat .pes/refs/heads/main
958591714ba40deea87807a2ff97615896fb83446b9ab6e78fb8e2dcfcfaf863
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$
```

Full integration test (make test-integration)

```
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$ make test-integration
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 02d1ea6297ee... Initial commit

Log after first commit:
commit 02d1ea6297ee2635416ab84e05a83277ac447630003c702e94723efa8079300e
Author: PES User <pes@localhost>
Date:   1776757214

    Initial commit


--- Second Commit ---
Committed: 1c823264e407... Update file.txt

--- Third Commit ---
Committed: 42201914184c... Add farewell

--- Full History ---
commit 42201914184cf7037941695bbd9a3b20f32d073e3c682ee102914fdad19d1492
Author: PES User <pes@localhost>
Date:   1776757214

    Add farewell

commit 1c823264e407520c507b41a7aef0f2a09d63e063ff666cf0ed0852406b534e54
Author: PES User <pes@localhost>
Date:   1776757214

    Update file.txt

commit 02d1ea6297ee2635416ab84e05a83277ac447630003c702e94723efa8079300e
Author: PES User <pes@localhost>
Date:   1776757214

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
42201914184cf7037941695bbd9a3b20f32d073e3c682ee102914fdad19d1492

--- Object Store ---
Objects created:
10
.pes/objects/02/d1ea6297ee2635416ab84e05a83277ac447630003c702e94723efa8079300e
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b5253611147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/1c/823264e407520c507b41a7aef0f2a09d63e063ff666cf0ed0852406b534e54
.pes/objects/42/201914184cf7037941695bbd9a3b20f32d073e3c682ee102914fdad19d1492
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
rishit@NAGASAI:~/PES1UG24AM225-pes-vcs$
```

Q5.1 — Branch creation & checkout

A branch in PES is stored as a file in .pes/refs/heads/ containing a commit hash.

- Creating a branch (pes branch <name>):
    - Create a new file .pes/refs/heads/<name>
    - Write the current HEAD commit hash into it
- Checkout (pes checkout <branch>):
    - Update .pes/HEAD to:

ref: refs/heads/<branch>

    - Read the commit hash from that branch file
    - Load the corresponding tree object
    - Update the working directory files to match that tree

Working directory update:

- Replace files with contents from the target commit's tree
- Remove files not present in target tree

Q5.2 — Detecting uncommitted changes

Answer:

The system compares:

- Current working directory files
  vs
- Index entries (stored metadata: hash, size, mtime)

Steps:

1. For each tracked file:
    - Compare file's current metadata (mtime, size)
    - If different → recompute hash

2. If hash differs from index → file is modified
3. If modified file differs across branches → checkout is refused

Reason:
To prevent overwriting user's uncommitted changes.

Q5.3 — Detached HEAD

Answer:

Detached HEAD means:

- .pes/HEAD contains a **commit hash directly**, not a branch reference

Effect:

- New commits are created but **not attached to any branch**
- They become "dangling" commits

Recovery:

- Create a branch pointing to that commit:

pes branch <name>

- Or manually update .pes/refs/heads/<name> with that commit hash

Q6.1 — Garbage Collection

Goal: Remove unreachable objects (blobs, trees, commits)

Algorithm:

1. Start from all branch heads (roots)
2. Traverse commits recursively:
    - commit → parent commit
    - commit → tree
    - tree → subtrees + blobs
3. Mark all visited objects as **reachable**
4. Delete all unmarked objects

Data structure:

- Use **Hash Set** to store visited object hashes

Q6.2 — GC race condition

If GC runs during a commit:

- GC may delete an object that a commit is about to reference
- This causes data corruption

Example:

1. Commit is writing new objects
2. GC runs and deletes “unreachable” objects
3. Commit tries to reference deleted object → failure

Git solution:

- Uses reference counting + reachability
- Uses temporary files and atomic renames
- GC avoids deleting recently created or in-use objects